

# 중간 보고서

## Pronimo : 3D 시각화 기반 영어 발음 교정 학습 서비스

(지정주제 : 다국어 발음 교정용 3D아바타 생성)



2026 세종대학교 컴퓨터공학과 캡스톤 1분반

팀 : 취업시켜조

컴퓨터공학과 21011610 조경태

22011629 강형민

23011714 정지태

21011632 권혁준

# 목 차

|                                              |    |
|----------------------------------------------|----|
| 1. 개발 목표                                     | 3  |
| 2. 팀원 역할 분담                                  | 4  |
| 3. 시스템 구조                                    | 5  |
| 3-1) 시스템 전체 구조                               | 5  |
| 3-2) Sub System별 상세 명세                       | 5  |
| 1. User Interface Sub System                 | 6  |
| 2. Backend API Sub System                    | 6  |
| 3. Curriculum & Learning Sub System          | 7  |
| 4. Media Upload Sub System                   | 7  |
| 5. AI Analysis Sub System                    | 8  |
| 6. Feedback & TTS Sub System                 | 8  |
| 7. Dashboard & Ranking Sub System            | 9  |
| 8. Database & Storage Sub System             | 9  |
| 4. 초기 기획 대비 수정 내용                            | 10 |
| 5. 현재 구현 내용                                  | 11 |
| 1. FE - 사용자 인터페이스 구현                         | 11 |
| 2. BE - 백엔드 서버                               | 12 |
| 3. AI                                        | 15 |
| 3-1. MediaPipe (Google AI Vision)            | 15 |
| 3-2. Azure Speech (Pronunciation Assessment) | 15 |
| 3-3. OpenAI GPT-4o-mini                      | 15 |
| 3-4. Supertone TTS                           | 15 |
| 4. AI 성능 향상을 위한 노력                           | 15 |
| 4-1. MediaPipe 데이터 경량화                       | 16 |
| 4-2. Viseme 시각 점수 보정                         | 16 |
| 4-3. LLM · TTS 파라미터 튜닝                       | 17 |

|                   |       |    |
|-------------------|-------|----|
| 5. 3D 모델링         | ..... | 18 |
| 6. 개발 추진 계획       | ..... | 20 |
| 7. 파트별 어려움 및 해결과정 | ..... | 22 |

## 1. 개발 목표

Pronimo는 3D 구강 구조 시각화와 AI 발음 분석 기술을 결합하여, 사용자가 영어 발음을 보다 직관적으로 학습하고 교정할 수 있도록 지원하는 영어 발음 교정 학습 플랫폼 개발을 목표로 합니다. 기존의 청각 중심 발음 학습 방식에서 벗어나, 발음 시 혀와 입술의 움직임을 3D로 시각화하여 학습자가 발음 원리를 직접 이해할 수 있는 학습 환경을 제공하고자 합니다.

또한 사용자의 음성 및 입모양 데이터를 기반으로 AI가 발음 정확도를 분석하고, 개인 맞춤형 피드백을 제공함으로써 반복 학습과 자기 주도 학습이 가능하도록 구현하는 것을 목표로 합니다. 특히 한국인 학습자가 어려워하는 /r/, /l/, /θ/, /ð/ 등의 발음을 효과적으로 교정할 수 있도록 지원하며, 단계별 학습, 오답노트, 학습 기록 관리 기능을 통해 학습 효과를 지속적으로 향상시키고자 합니다.

본 프로젝트는 React, Spring Boot, FastAPI, Blender, Three.js, MediaPipe 등 다양한 기술 스택을 활용하여 웹 기반 영어 발음 학습 환경을 구축하고자 합니다. 특히 단순 음성 평가 중심의 기존 발음 학습 방식에서 나아가, 3D 구강 구조 시각화와 AI 기반 발음 분석을 결합한 새로운 형태의 발음 학습 경험을 제공하는 것을 목표로 합니다. 또한 사용자의 음성 데이터와 입술 움직임 데이터를 함께 분석하여 보다 직관적이고 몰입감 있는 발음 교정 서비스를 구현하고자 합니다.

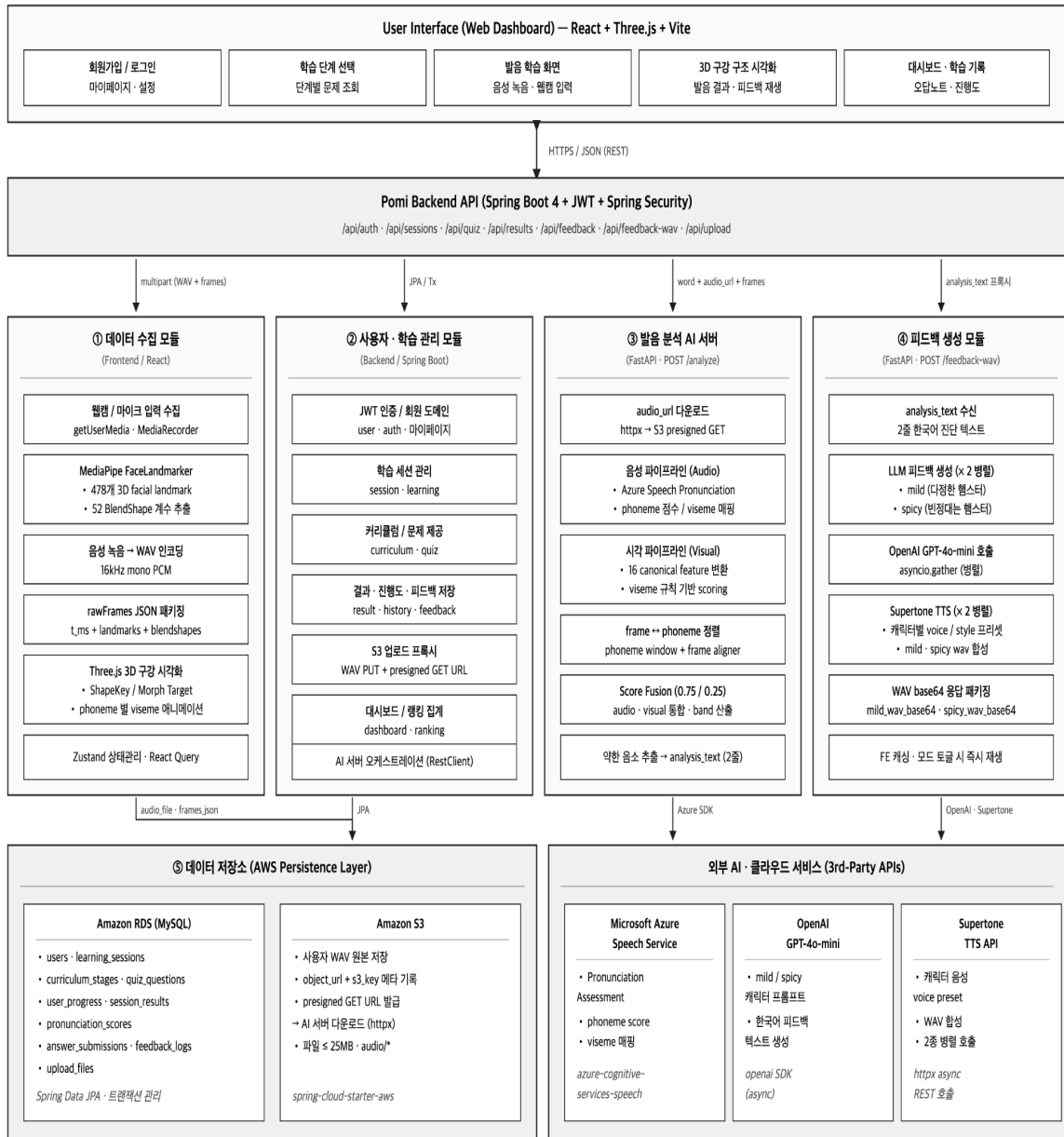
## 2. 팀원 역할 분담

- 1) 조경태 : AI 파트 - AI - FastAPI 서버 구현 / 음성 분석(Azure Speech PA) / 시각 분석(MediaPipe Viseme) / 점수 융합 및 약점 음소 추출 / LLM 피드백(GPT-4o-mini) + Supertone TTS
- 2) 강형민 : BE- spring / DB구현 / ews 서버 배포 / S3 배포 / FE - BE - AI 서버 연동

3) 정지태 : FE - react / ts 구현 및 vercel 배포 /git 형상관리 /web 구축

4) 권혁준 : 기획 / 3D 모델 - Blender / 디자인 - Figma

### 3. 시스템 구조



Pomi: 3D 시각화 기반 영어 발음 교정 학습 서비스 — 시스템 전체 구조 (System Architecture)

#### 3-1) 시스템 전체 구조

본 시스템은 영어 발음 학습과 발음 피드백 제공을 위한 통합 학습 솔루션으로, 음성 녹음, 입모양 데이터 수집, AI 발음 분석, 3D 발음 모델, 학습 통계 관리 기능을 제공합니다. 전체 시스템은 User Interface, Backend API, AI Analysis, Database & Storage, Dashboard & Ranking 등의 핵심 Sub System으로 구성되어 있으며, 이들은 유기적으로 연동되어 문제 풀이부터 발음 분석, 피드백 제공, 학습 기록 관리까지의 전 과정을 지원합니다.

### 3-2) Sub System별 상세 명세

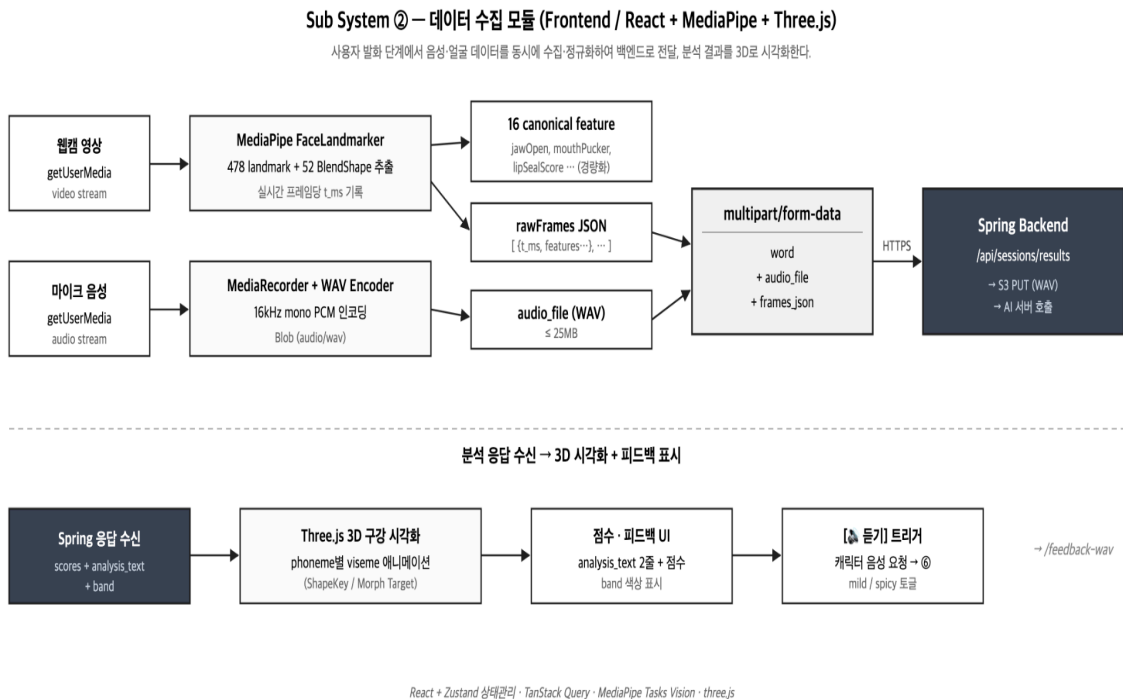
| No | Sub System                        | 상세 명세                                                                     |
|----|-----------------------------------|---------------------------------------------------------------------------|
| 1  | UI/UX                             | 사용자가 로그인, 단계별 학습, 음성 녹음, 결과 확인, 대시보드 조회를 수행하는 웹 화면이다.                     |
| 2  | Backend API<br>백엔드 API            | 프론트엔드 요청을 처리하고, DB·S3·AI 서버를 연결하는 중심 서버이다. 인증, 학습, 업로드, 분석, 통계 API를 제공한다. |
| 3  | Learning<br>학습                    | 단계별 문제와 학습 세션을 관리한다. 문제에는 정답 단어, 예문, 음소 정보, 3D 모델용 animationData가 포함된다.   |
| 4  | Media Upload<br>미디어 업로드           | 녹음된 음성 파일을 S3에 저장하고, 생성된 URL을 AI 서버 분석 요청에 사용한다.                          |
| 5  | AI Analysis<br>AI 분석              | 음성 URL과 MediaPipe 프레임을 받아 발음 점수, 입모양 점수, 종합 점수, 피드백 문장을 생성한다.             |
| 6  | Feedback<br>피드백                   | 분석 문장을 화면에 표시하고, 사용자가 요청하면 TTS 기반 음성 피드백을 제공한다.                           |
| 7  | Dashboard & Ranking<br>대시보드·랭킹    | 학습 기록을 기반으로 연속 학습일, 평균 점수, 자주 틀린 단어, 약한 음소, 랭킹 정보를 제공한다.                  |
| 8  | Database & Storage<br>데이터베이스·스토리지 | MySQL은 사용자·문제·학습 결과를 저장하고, AWS S3는 음성 파일을 저장한다.                           |

## 1. User Interface Sub System

사용자가 회원가입, 로그인, 단계별 학습, 음성 녹음, 결과 확인, 대시보드 조회를 수행하는 웹 화면이다.

- 입력: 로그인 정보, 선택한 문제, 녹음 음성, **MediaPipe** 얼굴 프레임 데이터
- 출력: 발음 점수, 음성/입모양 점수, 피드백 문장, **3D** 발음 모델 애니메이션, 학습 통계

## 2. Backend API Sub System



프론트엔드, AI 서버, MySQL, AWS S3를 연결하는 중심 서버이다.

- 처리: 인증, 커리큘럼 조회, 학습 세션 생성, 음성 업로드, AI 분석 요청, 결과 저장
- 반환: 분석 점수, 피드백 문장, 음성 피드백 데이터, 대시보드 통계, 랭킹 정보

### 3. Curriculum & Learning Sub System

단계별 발음 학습 문제와 학습 진행 상태를 관리하는 영역이다.

- Sentence Lv1 ~ Lv15 스테이지 제공, 정답 단어 · 예문 · 음소 · animationData 포함
- 학습 세션, 문항별 정답 여부, 선택 답안, 점수 결과를 저장

### 4. Media Upload Sub System

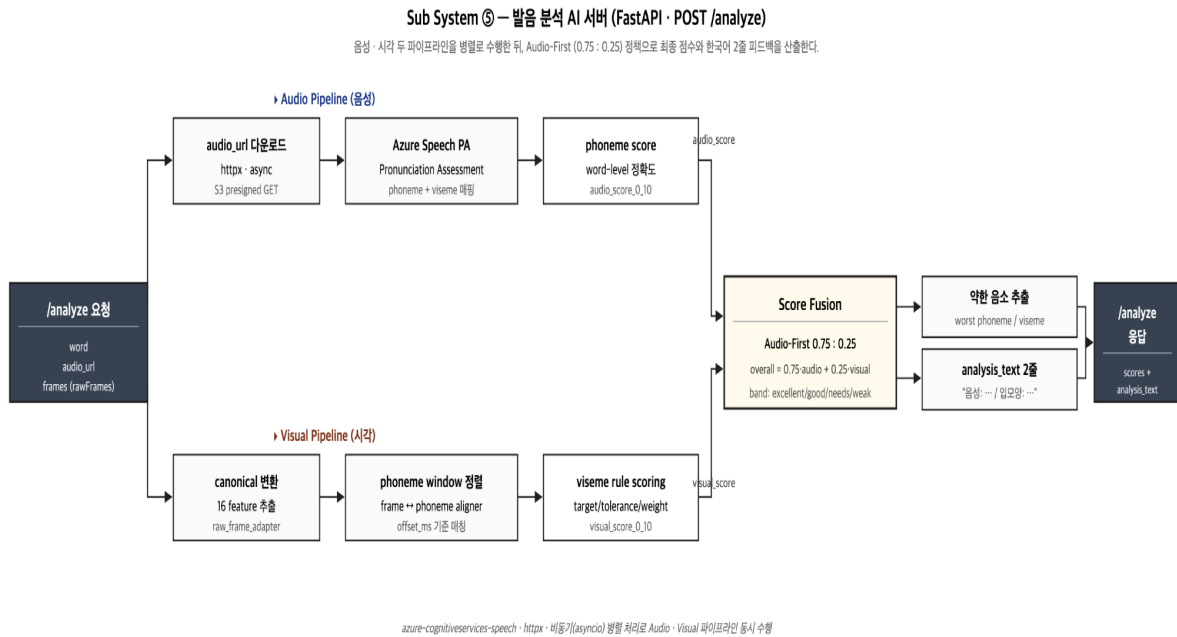
사용자가 녹음한 음성 파일을 처리하는 영역이다.

- 음성 파일을 AWS S3에 업로드하고, 파일 URL과 메타데이터를 DB에 저장
- 생성 값: uploadId, audioUrl, contentType, fileSize (AI 서버 분석 요청에 사용)

### 5. AI Analysis Sub System

발음 분석을 수행하는 AI 서버 연동 영역이다.

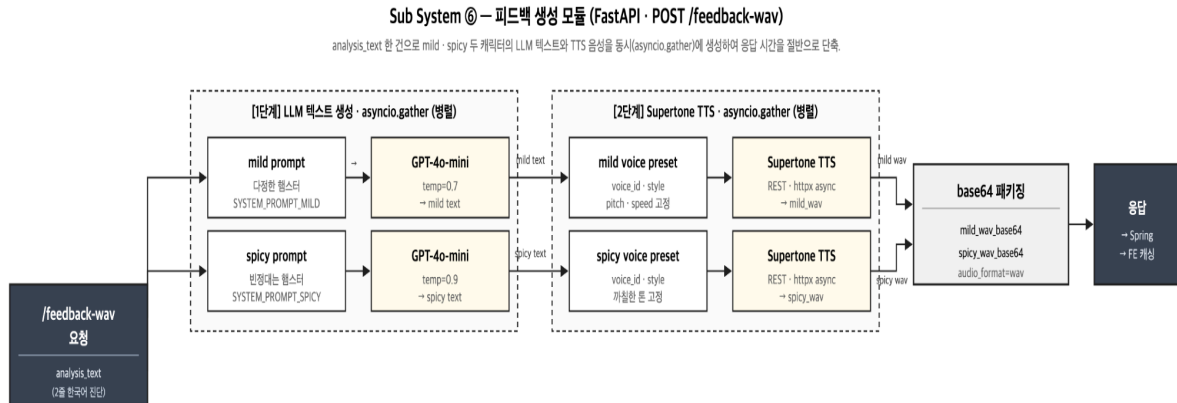




- 입력: 정답 단어, S3 음성 URL, MediaPipe 프레임 데이터
- 출력: overallScore, audioScore, visualScore, band, analysisText (결과 화면 및 기록에 사용)

## 6. Feedback & TTS Sub System

분석 결과를 사용자에게 이해하기 쉬운 피드백으로 제공하는 영역이다.



| 캐릭터 프리셋 정의                                                                                                                                                                                              | 병렬 처리 효과                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>mild</b> · 다정한 헬스더 · 존댓말 · 따뜻한 격려<br><small>↳ system_prompt + voice_id + style preset 모두 코드에 고정</small><br><br><b>spicy</b> · 빈정대는 헬스더 · 변명 · 허풍 · 까칠한 잔소리<br><small>↳ 점수와 무관하게 일정한 캐릭터 톤 유지</small> | <ul style="list-style-type: none"> <li>• <math>LLM \times 2 + TTS \times 2 =</math> 총 4건의 외부 API 호출</li> <li>• 직렬 호출 시 <math>(T_{LLM} \times 2) + (T_{TTS} \times 2) \approx 4 \cdot t</math></li> <li>• <b>asyncio.gather</b> 병렬 시 <math>\approx \max(T_{LLM}) + \max(T_{TTS}) \approx 2 \cdot t</math></li> </ul> ⇒ 전체 응답 시간 약 50% 단축, FE는 코드 토크 시 즉시 재생 |

- 기본 피드백: AI 서버가 반환한 **analysisText**를 화면에 즉시 표시
- 듣기: /feedback-wav API 호출 → mild\_wav\_base64, spicy\_wav\_base64, audio\_format 수신 후 WAV로 변환 재생

## 7. Dashboard & Ranking Sub System

사용자의 학습 기록을 통계로 가공하는 영역이다.

- 집계: 최근 4주 활동 히트맵, 주간 평균 점수, 학습 연속일, 자주 틀린 단어, 약한 음소, 랭킹
- /api/dashboard/summary로 한 번에 전달하여 대시보드 화면 구성

## 8. Database & Storage Sub System

서비스 운영에 필요한 데이터를 저장하는 영역이다.

- **MySQL**: 사용자 정보, 커리큘럼, 문제, 학습 세션, 제출 결과, 점수, 피드백 로그, 업로드 메타데이터
- **AWS S3**: 녹음 음성 파일 저장 (DB에는 S3 URL과 메타데이터만 저장)

## 4. 초기 기획 대비 수정 내용

### 1. 단순 발음 학습 → 기본 발음 훈련 기능 추가

초기에는 단어 발음 중심 학습만 고려하였으나, 한국인 학습자들이 특정 음소 자체를 어려워한다는 점을 반영하여 기본 발음 연습 기능을 추가하였습니다.

특히 /r/, /l/, /th/, /f/, /v/ 등 한국인이 어려워하는 발음을 중심으로 음소 단위 학습 기능을 구성하도록 수정하였습니다.

### 2. 단순 점수 제공 → 학습 기록 및 통계 기능 추가

초기에는 문제별 점수만 제공하는 구조였으나, 사용자의 학습 지속성을 높이기 위해 학습 기록 기능을 추가하였습니다.

이에 따라 연속 학습 일수, 월별 학습 기록(잔디 형태), 요일별 점수 그래프, 사용자 간 월간 랭킹 기능 등을 추가하는 방향으로 기획을 확장하였습니다.

### 3. 단순 발음 학습 → 유사 음소 간 비교 학습 기능 추가

초기에는 개별 발음을 단순 반복 학습하는 방식만 고려하였으나, 실제 한국인 학습자들은 서로 유사한 영어 발음을 구분하는 과정에서 어려움을 겪는 경우가 많다는 점을 반영하여 유사 음소 비교 학습 기능을 추가하였습니다.

특히 /r/과 /l/, /b/와 /v/, /s/와 /sh/ 등 혼동하기 쉬운 발음을 서로 비교하며 학습할 수 있도록 구성하였으며, 각 발음별 입술과 혀의 움직임 차이를 3D 구강 구조를 통해 시각적으로 확인할 수 있도록 기획 방향을 수정하였습니다.

### 4. 3D 단면 시각화 중심 구조 → 단면 + 정면 동시 시각화 구조로 확장

초기에는 사용자가 발음 시 혀와 구강 내부 움직임을 확인할 수 있도록 3D 단면 구조만 제공하는 방향으로 기획하였습니다. 그러나 실제 발음 학습에서는 입술 모양과 외형적인 움직임 또한 매우 중요한 요소라고 판단하였습니다.

이에 따라 내부 구강 구조를 보여주는 단면 시각화와 함께, 사용자의 입술 모양과 발음 형태를 직관적으로 확인할 수 있는 정면 시각화 구조를 추가하였습니다. 이를 통해 사용자는 발음 시 내부적인 혀 움직임과 외부적인 입술 형태를 동시에 확인할 수 있도록 서비스 구조를 수정하였습니다

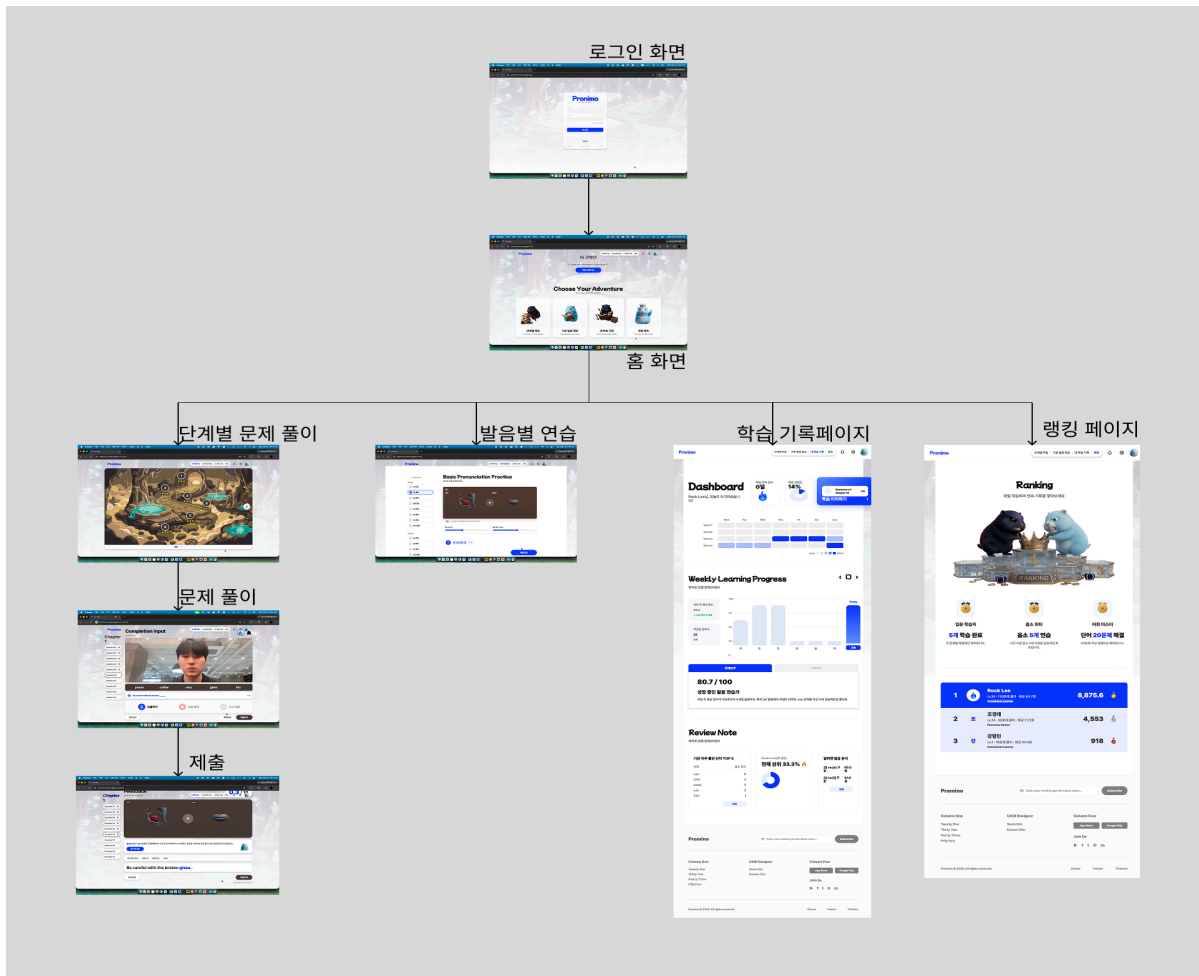
## 5. 현재 구현 내용

### 1. FE - 사용자 인터페이스 구현

사용자 인터페이스는 **FastAPI** 기반의 백엔드와 **React** 기반의 프론트엔드를 연동하는 방식으로 구성하였다. 현재 백엔드에는 관리자 인증을 위한 로그인 기능과 카메라의 등록, 수정, 조회 기능 등 장치 관리에 필요한 주요 엔드포인트가 구현되어 있다.

프론트엔드에서는 이러한 백엔드 기능을 기반으로 관리자가 시스템에 접속하여 카메라 및 관련 정보를 효율적으로 관리할 수 있도록 웹 대시보드 형태의 인터페이스를 구성하고 있다. 또한 발음 데이터와 **3D** 입모양 모델을 연동하는 데모를 구현함으로써, 향후 음성 데이터와 발음 타이밍 정보를 활용한 시각화 기능으로 확장할 수 있는 기반을 마련하였다.

User Flow (구현 ui)



## 2. BE

### 백엔드 서버

백엔드 서버는 사용자의 발음 학습 흐름을 관리하고, 프론트엔드와 AI 서버 사이의 중간 처리 계층 역할을 수행한다. 사용자의 회원가입 및 로그인, 학습 커리큘럼 조회, 학습 세션 관리, 음성 파일 업로드, AI 발음 분석 요청, 분석 결과 저장, 대시보드 통계 제공 등의 기능을 담당한다.

프론트엔드에서 사용자가 발음 녹음을 완료하면 백엔드는 음성 파일을 AWS S3에 업로드하고, 해당 파일에 접근 가능한 **presigned URL**을 생성한다. 이후 정답 단어, S3 음성 URL, MediaPipe 얼굴 프레임 데이터를 AI 서버로 전달하여 발음 분석을 요청한다. AI

서버로부터 반환된 음성 점수, 입모양 점수, 종합 점수, 피드백 문장은 **DB**에 저장되며, 프론트엔드에 응답되어 사용자에게 즉시 제공된다.

## 주요 기능

### 사용자 인증 및 계정 관리

- 이메일 기반 회원가입 및 로그인 기능을 제공한다.
- 로그인 성공 시 **JWT**를 발급하여 인증이 필요한 **API** 요청에 사용한다.
- 사용자의 이름, 닉네임, 학습 레벨 등의 정보를 관리한다.

### 커리큘럼 및 문제 관리

- **Sentence Lv1**부터 **Sentence Lv15**까지의 학습 단계를 구성하였다.
- 각 단계는 **10**개의 문장형 발음 문제로 구성되며, 전체 **150**개의 문제 데이터를 저장한다.
- 각 문제는 정답 단어, 대표 음소, 난이도, **3D** 발음 모델 구동을 위한 **animationData**를 포함한다.

### 학습 세션 관리

- 사용자가 학습을 시작하면 **LearningSession**을 생성한다.
- 세션 내에서 사용자가 푼 문제와 제출 결과를 **SessionResult**로 저장한다.
- 기존 세션이 선택한 레벨과 맞지 않을 경우 새 세션을 생성하도록 처리하여 잘못된 문제 재사용을 방지하였다.

### 음성 업로드 및 AI 분석 연동

- 프론트엔드에서 전달한 **WAV** 음성 파일을 **AWS S3**에 저장한다.
- **S3 presigned URL**을 발급하여 **AI** 서버가 음성 파일을 다운로드할 수 있도록 한다.

- 정답 단어, 음성 URL, MediaPipe 프레임 데이터를 AI 서버 /analyze 엔드포인트로 전달한다.
- AI 서버 응답의 점수와 피드백 문장을 저장하고 프론트엔드에 반환한다.

#### 피드백 음성 생성

- 사용자가 피드백 음성을 듣고자 할 경우 /api/media/feedback-wav API를 통해 AI 서버 /feedback-wav를 프록시 호출한다.
- AI 서버는 분석 문장을 기반으로 mild/spicy 두 가지 캐릭터 음성을 생성한다.
- 백엔드는 반환된 wav base64 데이터를 프론트엔드에 전달하고, 프론트엔드는 이를 오디오로 재생한다.

#### 대시보드 통계 제공

- 사용자의 학습 기록을 기반으로 대시보드 요약 데이터를 제공한다.
- 최근 4주 학습 히트맵, 주간 점수 비교, 자주 틀린 단어, 약한 음소, 종합 피드백을 계산한다.
- 프론트엔드는 /api/dashboard/summary 한 번의 호출로 대시보드에 필요한 데이터를 받을 수 있다.

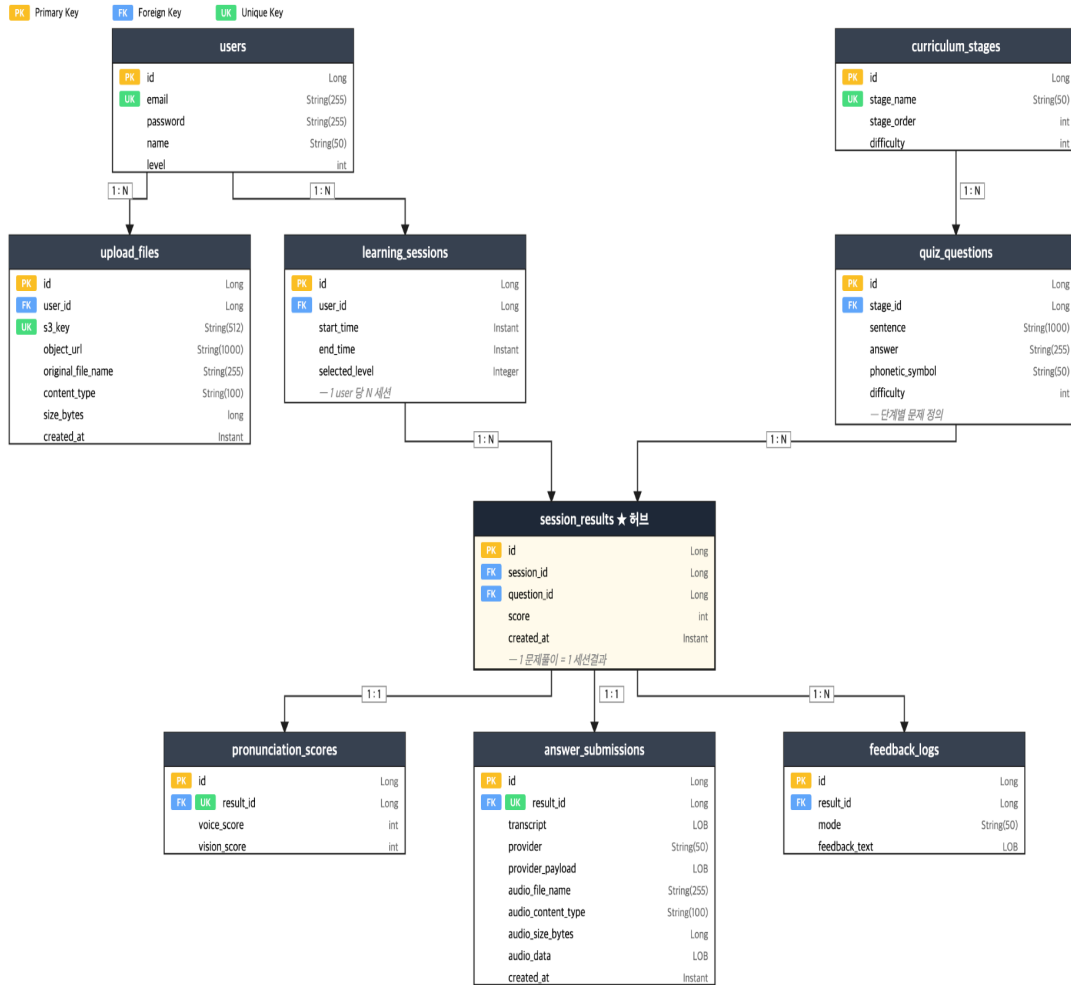
#### 저장소 처리 모듈

백엔드는 MySQL을 기반으로 사용자, 커리큘럼, 학습 세션, 문제 풀이 결과, 음성 업로드 정보, AI 분석 결과, 피드백 로그를 저장한다. 음성 파일 자체는 DB에 저장하지 않고 AWS S3에 저장하며, DB에는 S3 key, URL, contentType, size 등의 메타데이터만 저장한다. 주요 테이블은 다음과 같다.



## Pomi 데이터베이스 ERD — 9개 핵심 테이블

음성 파일 저장 · AI 분석 결과 관리 · 대시보드 통계 계산을 위한 관계형 구조



### 설계 의도

- users · curriculum\_stages 두 축에서 모든 학습 데이터가 파생되어, 사용자별 단계별 통계를 그대로 집계할 수 있다.
- session\_results 가 허브 역할을 맡아 점수(pronunciation\_scores) · 원본 응답(answer\_submissions) · 피드백(feedback\_logs) 을 1:1 / 1:N 으로 묶는다.
- upload\_files 는 S3에 업로드한 WAV의 메타데이터를, answer\_submissions.audio\_data 는 AI 분석 원본 raw payload를 보존한다.

이 구조를 통해 음성 파일 저장, AI 분석 결과 관리, 대시보드 통계 계산이 가능하도록 설계하였다.

### 3.AI

본 프로젝트는 단일 모델을 직접 학습하는 대신 검증된 사전학습 AI 서비스 4가지를 파이프라인으로 결합한 사전학습 AI 결합형 시스템으로 설계되었다. 각 모델은 독립적인 역할을 가지며, 결과는 FastAPI 기반 AI 서버에서 통합·해석된다.

#### 3-1. MediaPipe (Google AI Vision)

MediaPipe는 웹캠 영상으로부터 얼굴 랜드마크 478개(3D 좌표)와 블렌드셰이프 계수 52개를 실시간으로 추출하는 사전학습 모델이다. 본 시스템에서는 발음 시 입술과 턱의 움직임을 정량적으로 측정하기 위해 클라이언트(브라우저)에서 직접 실행되며, 추출된 프레임 데이터는 시각 분석 단계의 입력으로 사용된다.

#### 3-2. Azure Speech (Pronunciation Assessment)

Microsoft Azure Speech SDK가 제공하는 Pronunciation Assessment 기능을 활용하여 사용자가 녹음한 음성을 phoneme(음소) 단위로 평가한다. 단어·음소별 정확도 점수와 함께 시각 분석과의 동기화를 위한 viseme 매핑 정보, 타이밍 정보(offset, duration)를 함께 반환받아 음성 파이프라인의 핵심 결과로 사용한다.

#### 3-3. OpenAI GPT-4o-mini

점수와 통계만으로는 학습자가 어떤 방향으로 교정해야 할지 직관적으로 이해하기 어렵기 때문에, GPT-4o-mini를 사용하여 분석 결과(analysis\_text)를 자연어 피드백으로 변환한다. 본 시스템은 두 가지 캐릭터(다정한 햄스터 / 빈정대는 햄스터)에 해당하는 별도의 system prompt를 병렬로 호출하여, 동일한 발음 결과에 대해 두 가지 톤의 피드백을 동시에 생성한다.

#### 3-4. Supertone TTS

GPT가 생성한 텍스트 피드백을 음성으로 들려주기 위해 Supertone TTS API를 연동하였다. 캐릭터별로 별도의 voice ID와 style preset을 지정하여 mild 캐릭터는 따뜻한 격려의 톤,

spicy 캐릭터는 까칠한 잔소리의 톤으로 음성을 합성하며, 두 캐릭터의 wav는 한 번의 요청으로 병렬 생성되어 프론트엔드에 base64 형태로 전달된다.

#### 4. 성능 향상을 위한 노력

사전학습 모델을 그대로 결합하는 구조 특성상, 모델 자체의 성능보다는 데이터 흐름의 효율성과 결과의 품질·일관성을 높이는 데 초점을 맞춰 다음 세 가지 방향으로 개선을 진행하였다.

##### 4-1. MediaPipe 데이터 경량화를 통한 전송 비용 절감

MediaPipe가 매 프레임 출력하는 raw 데이터는 478개의 3차원 landmark와 52개의 blendshape 계수로 구성되어 있어, 이를 그대로 백엔드로 전송할 경우 네트워크 대역폭과 서버 처리 비용이 크게 증가한다. 이를 해결하기 위해 프론트엔드 단계에서 발음 분석에 실제로 필요한 16개의 canonical feature(jawOpen · mouthFunnel · mouthPucker · lipSealScore · outerLipHeightNorm 등)만을 추출·정규화하여 전송하도록 변경하였다. 그 결과 페이로드 크기를 대폭 축소하였으며, 백엔드 또한 별도의 전처리 부담 없이 곧바로 시각 분석 단계로 진입할 수 있게 되었다.

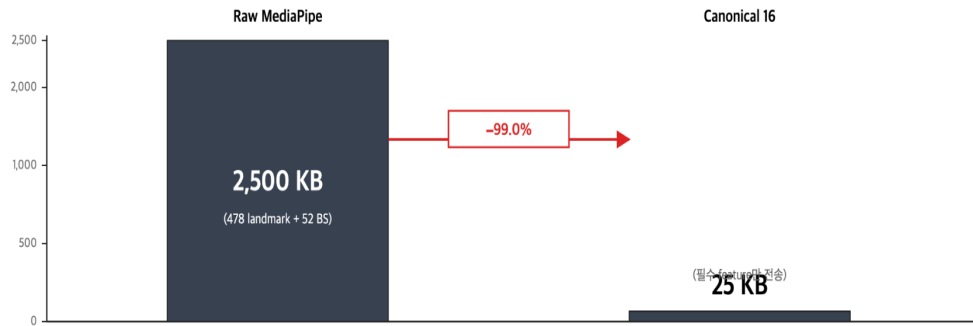
#### [성능 개선 4-1] MediaPipe 데이터 경량화 — Raw vs Canonical 16 Feature

프론트엔드에서 발음 분석에 필요한 16개 feature만 추출 전송하여 네트워크 페이로드를 약 99% 절감

[표] 단계별 페이로드 크기 비교

| 측정 단위              | Raw MediaPipe                             | Canonical 16                 | 절감률    |
|--------------------|-------------------------------------------|------------------------------|--------|
| 프레임당 raw 데이터       | $478 \times 3 \times 4 = 5,736 \text{ B}$ | $16 \times 4 = 64 \text{ B}$ | -98.9% |
| + blendshape 52개   | + 208 B = 5,944 B                         | (canonical에 통합)              | —      |
| 초당 (30 fps 기준)     | 178.3 KB                                  | 1.9 KB                       | -98.9% |
| 5초 발화 (150 frames) | 870 KB                                    | 9.4 KB                       | -98.9% |
| JSON 직렬화 후 (실측)    | ≈ 2.5 MB                                  | ≈ 25 KB                      | -99.0% |

[그래프] 5초 발화 기준 페이로드 크기 (JSON 직렬화 후, 단위: KB)



네트워크 전송 데이터를 거의 100배 줄여 모바일·저속 환경에서도 안정적인 분석 응답을 확보하였다.

## 4-2. Viseme 시각 점수 보정 (Tolerance · Weight 튜닝)

시각 분석은 phoneme 시간 구간에 해당하는 프레임에서 canonical feature를 통계적으로 비교하여 viseme별 점수를 산출하는 규칙 기반(rule-based) 방식으로 구현되었다. 초기에는 3D Talking Head 모델에서 추출한 reference feature를 그대로 사용하였으나, 실제 사용자 발음에서는 점수 분포가 한쪽으로 치우치거나 시각적으로 명확한 차이를 잡아내지 못하는 문제가 발생하였다. 이를 해결하기 위해 viseme별로 target / tolerance / weight 파라미터를 수작업으로 조정하고 visual\_reliability 가중치를 도입하여, 시각적으로 잘 보이는 viseme(aa, PP, FF 등)에는 큰 가중치를, 구분이 어려운 자음에는 낮은 가중치를 부여하는 방식으로 점수 신뢰도를 개선하였다.

#### [성능 개선 4-2] Viseme 시각 점수 보정 – Tolerance · Reliability · Fusion 적용 전/후

0점 수렴 비율을 32%p 낮추고, 시각 정보로 판단 불가능한 음소가 다수인 단어에서도 합리적인 점수를 산출

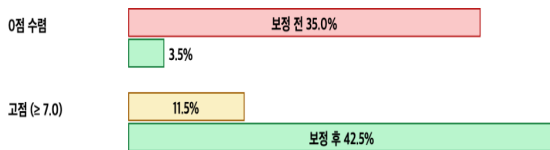
[표 1] 동일 발음 샘플에 대한 시각 점수 비교 (10점 만점)

| 테스트 단어 | 주요 viseme    | 보정 전 visual_score | 보정 후 visual_score | 개선 요인                                     |
|--------|--------------|-------------------|-------------------|-------------------------------------------|
| think  | TH · kk      | 0.0               | 5.5               | 시각 판단 불가 viseme → visual_reliability ↓ 적용 |
| apple  | aa · PP      | 4.2               | 8.4               | tolerance 확장 + 개안차 보정                     |
| right  | RR · DD      | 1.1               | 6.5               | 혀 의존 viseme 가중치 하향                        |
| five   | FF · aa      | 6.0               | 8.7               | visible viseme 가중치 상향                     |
| mother | PP · TH · NN | 0.5               | 5.2               | PP만 시각 반영, 나머지는 음성으로 융합                   |

[표 2] 200건 샘플 기준 시각 점수 분포 비교

| 구간             | 보정 전 비율 | 보정 후 비율 | 변화       |
|----------------|---------|---------|----------|
| 0점 수렴 (≤ 1.0)  | 35.0 %  | 3.5 %   | -31.5 %p |
| 저점 (1.0 ~ 4.0) | 28.0 %  | 12.0 %  | -16.0 %p |
| 중간 (4.0 ~ 7.0) | 25.5 %  | 42.0 %  | +16.5 %p |
| 고점 (≥ 7.0)     | 11.5 %  | 42.5 %  | +31.0 %p |

[그래프] 보정 전/후 점수 분포 시각화



※ visual\_score 0점 수렴 비율이 35.0% → 3.5%로 31.5%p 감소, 고점 비율이 11.5% → 42.5%로 31.0%p 증가하여 점수 분포가 합리적으로 재배치되었음.

### 4-3. LLM · TTS 파라미터 튜닝을 통한 캐릭터 일관성 확보

GPT-4o-mini와 Supertone TTS는 모두 외부 사전학습 서비스이므로 별도의 학습 없이 프롬프트·파라미터 튜닝만으로 원하는 결과 품질을 확보해야 한다. LLM 측에서는 캐릭터별로 별도의 **system prompt**(말투·호칭·금칙어)와 **temperature·top\_p** 값을 조정하여 **mild/spicy** 두 캐릭터의 톤이 점수에 따라 흔들리지 않고 일관되게 유지되도록 하였다. TTS 측에서는 캐릭터별 **voice ID**와 **style preset, pitch·speed** 값을 고정하여 동일 캐릭터의 음색이 매 회 동일하게 유지되도록 설계하였다. 또한 LLM 두 건과 TTS 두 건을

asyncio.gather 로 묶어 병렬 호출함으로써 전체 피드백 응답 시간을 절반 수준으로 단축하였다.

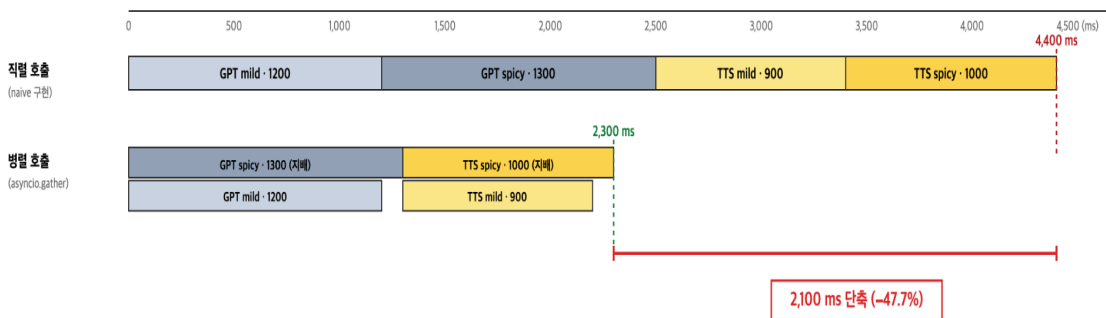
#### [성능 개선 4-3] LLM · TTS 직렬 호출 vs asyncio.gather 병렬 호출

LLM × 2 + TTS × 2 = 총 4건의 외부 API 호출을 2단계 병렬화하여 응답 시간을 약 47% 단축

[표] /feedback-wav 단계별 응답 시간 측정 (ms, 30회 평균)

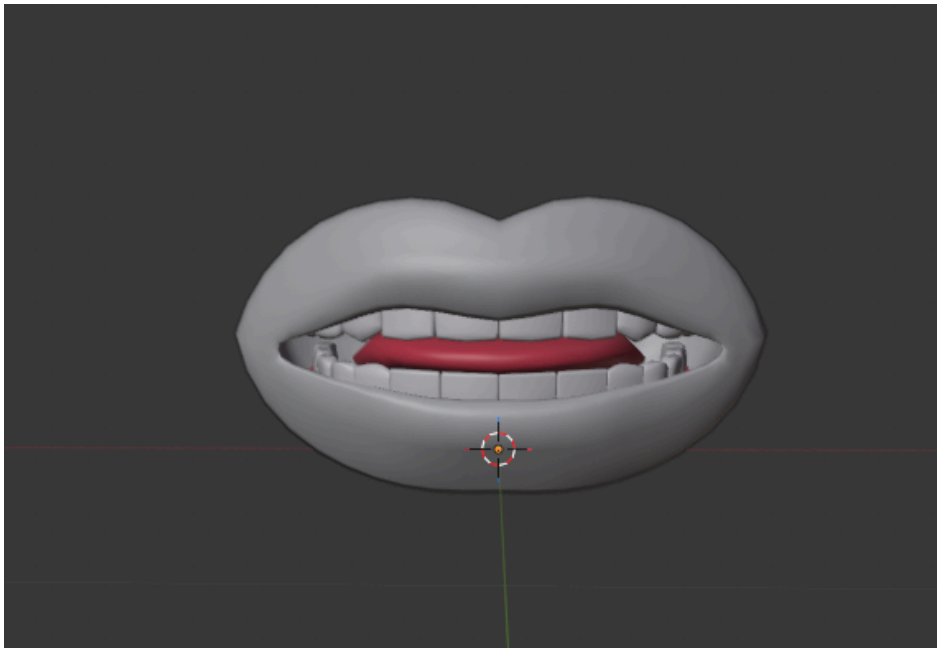
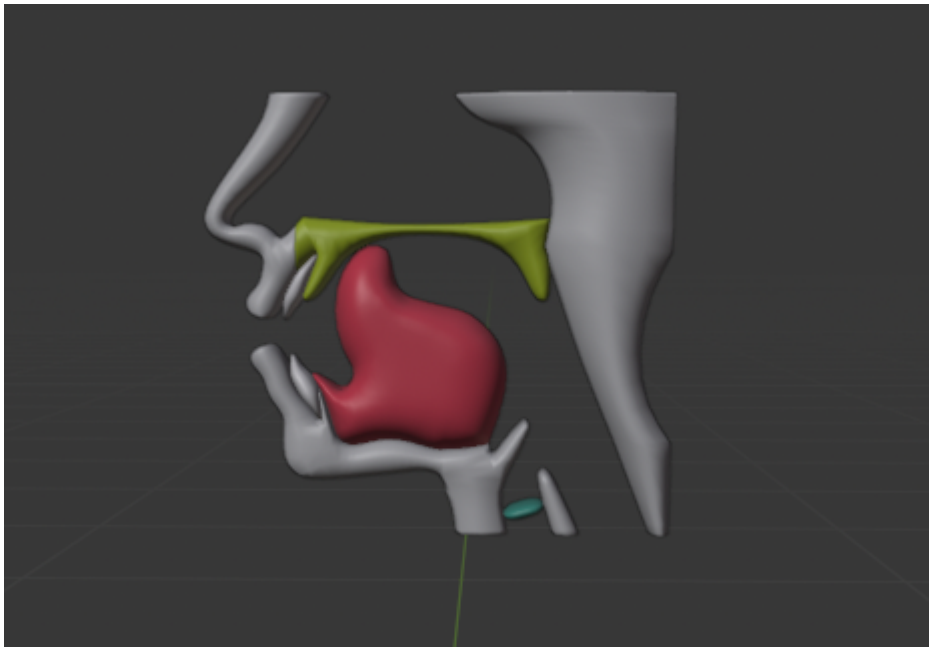
| 처리 단계                 | 개별 호출 시간 | 직렬 누적 (ms) | 병렬 누적 (ms)   | 비고                       |
|-----------------------|----------|------------|--------------|--------------------------|
| GPT-4o-mini (mild)    | 1,200    | 1,200      | ┘            | asyncio.gather 1단계 진입    |
| GPT-4o-mini (spicy)   | 1,300    | 2,500      | <b>1,300</b> | max(1200, 1300) = 1,300  |
| Supertone TTS (mild)  | 900      | 3,400      | ┘            | asyncio.gather 2단계 진입    |
| Supertone TTS (spicy) | 1,000    | 4,400      | <b>2,300</b> | + max(900, 1000) = 2,300 |
| 총 응답 시간               | —        | 4,400 ms   | 2,300 ms     | -47.7 % (약 절반)           |

[그래프] 호출 타임라인 — 직렬 vs 병렬 (1 칸 = 100 ms)



LLM 단계와 TTS 단계 각각에 asyncio.gather 를 적용하여 동일 단계 내 두 캐릭터 호출이 동시에 진행되도록 처리하였다.

## 5. 3D 모델링



현재 Pronimo 프로젝트의 3D 구강 구조 모델링 작업은 Blender를 기반으로 구현을 완료한 상태입니다. 발음 학습에 필요한 혀, 입술, 치아 등의 구조를 모델링하였으며, 영어 발음 학습에 활용할 수 있도록 구강 내부 단면 구조와 정면 입술 구조를 함께 구성하였습니다. 또한 발음별 입모양과 혀 움직임을 표현하기 위해 **Shape Key** 기반 발음 포즈를 제작하였으며, 이를 통해 특정 음소 발음 시 구강 구조가 자연스럽게 변화할 수 있도록 구현하였습니다. 특히 /r/, /l/, /f/, /v/, /θ/ 등 한국인 학습자가 어려워하는 발음을 중심으로 주요 발음 포즈를 우선 제작하였습니다.

추가적으로 웹 환경에서의 실시간 렌더링을 고려하여 모델 최적화 작업도 함께 진행하였습니다. 초기 모델링 과정에서 발생한 과도한 메쉬 면 수를 조정하고, 발음 표현에 중요한 영역 중심으로 디테일을 유지하는 방식으로 수정하여 **Three.js** 기반 웹 환경에서도 활용 가능하도록 구현하였습니다.

현재 **Blender**에서 제작한 **3D** 모델은 **glb** 형식으로 변환되어 웹 환경과 연동이 완료된상태이며, 웹 상에서 발음별 구강 구조를 시각적으로 확인할 수 있도록 구현하였습니다.



## 6. 개발 추진 계획

[illegible]

## 추가 계획

### 1. AI 분석 연동 안정화

AI 서버의 /analyze, /feedback-wav 응답 구조에 맞춰 백엔드 DTO와 프론트엔드 결과 화면을 최종 정리한다. 발음 점수, 입모양 점수, 피드백 문장이 안정적으로 표시되는지 검증한다.

### 2. 배포 환경 안정화

Vercel 프론트엔드와 EC2 백엔드 간 CORS, HTTPS, API 통신 문제를 점검하고, 배포 서버에서 DB와 AI 서버 연동이 정상 동작하는지 확인한다.

### 3. 테스트 및 오류 수정

실제 사용 흐름 기준으로 회원가입, 로그인, 문제 풀이, 음성 업로드, AI 분석, 결과 저장, 대시보드 조회까지 통합 테스트를 진행한다. 발견된 오류는 로그와 DB 데이터를 기반으로 수정한다.

## 7. 파트별 어려움 및 해결과정

### 1. 음성 파일 처리 및 AI 서버 연동

프론트엔드에서 전달되는 음성 파일을 AI 서버에 전달할 경우 요청 크기가 커지고, 백엔드와 AI 서버 간 데이터 전달 방식이 복잡해지는 문제가 있었다.

이를 해결하기 위해 백엔드가 음성 파일을 AWS S3에 업로드하고, AI 서버에는 **presigned URL**만 전달하는 방식으로 구조를 변경하였다. AI 서버는 해당 URL을 통해 음성 파일을 다운로드하여 분석을 수행한다. 이 방식으로 백엔드는 파일 바이너리를 직접 들고 다니지 않아도 되고, AI 서버와의 연동도 JSON 기반으로 단순화할 수 있었다.

### 2. DB 컬럼 크기 문제

AI 서버의 분석 결과와 피드백 내용을 저장하는 과정에서 **provider\_payload**, **feedback\_text**, **animation\_data**와 같은 컬럼에 긴 JSON 또는 문장이 저장되었다. 기존 컬럼 크기로는 데이터를 저장하지 못해 **Data too long for column** 오류가 발생하였다.

이를 해결하기 위해 해당 컬럼들을 **LONGTEXT** 타입으로 변경하였다. 또한 이후 응답 구조를 슬림화하여 불필요하게 큰 데이터를 저장하지 않도록 수정하였다.

### 3. 대시보드 통계 집계

대시보드에서는 최근 4주 활동, 주간 점수 비교, 자주 틀린 단어, 약한 음소, 종합 피드백 등 여러 데이터를 한 화면에서 필요로 했다. 이를 프론트엔드가 여러 API로 나누어 호출하면 로딩 상태 관리가 복잡해지는 문제가 있었다.

이를 해결하기 위해 `/api/dashboard/summary` 하나의 API에서 대시보드에 필요한 모든 데이터를 내려주도록 구성하였다. 백엔드는 저장된 학습 결과를 기반으로 통계를 계산하고, 프론트엔드는 한 번의 호출로 대시보드 화면을 구성할 수 있게 되었다.

#### 4. 발음 데이터와 3D 입모양 모델의 연동 문제

프로젝트를 진행하는 과정에서 가장 어려웠던 부분 중 하나는 발음 데이터와 3D 입모양 모델의 셰이프키(Shape Key)를 정확하게 연결하는 작업이었다. 초기에는 발음 기호와 모델 내 셰이프키의 대응 관계가 명확하지 않아, 데이터는 정상적으로 전달되더라도 화면상에서 입모양 변화가 제대로 나타나지 않는 문제가 발생하였다. 이를 해결하기 위해 모델에 포함된 셰이프키 목록을 직접 확인하고, 각 발음 기호를 대응되는 **viseme** 형태로 매핑하는 구조를 설계하였다. 이후 발음 데이터가 들어오면 해당 **viseme** 값을 기준으로 3D 모델의 셰이프키를 구동하도록 수정하여 문제를 해결하였다.

#### 5. 발음 재생 속도 및 입모양 전환 타이밍 조정 문제

또 다른 어려움은 발음 예시와 입모양 애니메이션의 재생 속도 및 전환 타이밍을 자연스럽게 맞추는 것이었다. 초기 구현에서는 애니메이션 속도가 빠르게 느껴져 사용자가 입모양 변화를 직관적으로 확인하기 어려웠고, 재생 종료 후에도 입모양이 중립 상태로 복귀하지 않는 문제가 있었다. 이를 해결하기 위해 속도 조절 범위를 재설정하여 더 느린 재생이 가능하도록 수정하였으며, 재생 종료 시 모든 **viseme** 값을 초기화하여 중립 상태로 돌아가도록 로직을 보완하였다. 이를 통해 사용자 입장에서 보다 자연스럽게 확인하기 쉬운 데모 환경을 구현할 수 있었다.

#### 6. 3D 모델링 구현 과정에서의 어려움과 해결 과정

Pronimo 프로젝트에서는 Blender를 활용하여 발음 학습용 3D 구강 구조 모델을 제작하였습니다. 그러나 일반적인 3D 캐릭터 모델링과 달리, 발음 학습용 모델은 혀와 입술의 움직임을 학습자가 직관적으로 이해할 수 있어야 한다는 어려움이 있었습니다. 특히 /r/, /l/, /θ/, /ð/ 등 한국인이 어려워하는 발음은 혀 위치와 입모양 차이가 미세하여 이를 3D 모델에서 자연스럽게 명확하게 표현하는 과정이 쉽지 않았습니다. 초기에는 구강 내부 단면 구조만 제공하려 하였으나, 단면만으로는 입술의 형태 변화를 충분히 보여주기 어렵다고 판단하여 정면 입술 구조까지 함께 시각화하는 방향으로 수정하였습니다. 또한 초기 모델은 디테일을 높이는 과정에서 메쉬의 면 수(Polygon)가 지나치게 많아져 웹 환경에서 렌더링 성능이 저하되는 문제가 발생하였습니다. 이를 해결하기 위해 학습에 핵심적인 혀와 입술 부분은 디테일을 유지하고, 불필요한 영역은 단순화하는 방식으로 모델 최적화를 진행하였습니다. 마지막으로 모든 발음을 개별 애니메이션으로 제작하는 것은 비효율적이라고 판단하여, Blender의 Shape Key 기능을 활용해 기본 발음 포즈를 제작하고 이를 조합하여 단어 발음을 표현하는 방식으로 구현 방향을 수정하였습니다. 이를 통해 웹 환경에서도 비교적 자연스럽게 효율적인 발음 애니메이션 구현이 가능하도록 개선하였습니다.

## 7. Viseme 시각 점수의 두 가지 한계와 해결 과정 (AI 파트)

본 시스템의 시각 분석 모듈은 phoneme별로 정의된 viseme(입 모양)의 기준값(target)과 사용자 프레임에서 측정된 canonical feature를 비교하여 규칙 기반(rule-based)으로 점수를 산출하는 구조입니다. AI 파트의 가장 큰 어려움은 이 시각 점수가 의도와 달리 0점으로 수렴하거나, 애초에 시각 정보만으로는 판단이 불가능한 음소가 다수 존재한다는 점이었습니다. 두 한계의 원인과 해결 과정을 다음과 같이 정리합니다.

### 7-1. 한계 ① — Viseme 점수가 0점으로 수렴하는 문제

초기 구현에서는 viseme별 target / tolerance를 단일 값으로 설정하고, 사용자의 observed feature가  $(target \pm tolerance)$  구간을 벗어나면 점수가 급격히 0으로 떨어지도록

정의하였습니다. 그러나 실제 사용자 데이터를 적용한 결과, 다음과 같은 경우 점수가 비합리적으로 0에 수렴하는 현상이 반복적으로 발생하였습니다.

- 사용자의 입 크기·턱 길이 등 개인차로 인해 **reference**로 사용한 **3D Talking Head**의 **feature** 분포와 절대값에 차이가 큰 경우
- 카메라 각도·조명 변화로 일부 **landmark**가 흔들려, 한 프레임의 **outlier** 값이 **phoneme** 구간 전체의 통계값을 왜곡한 경우
- /p/, /t/ 처럼 발음 지속 시간이 매우 짧은 음소에서 해당 **phoneme** 구간에 포함되는 프레임 수가 부족하여 통계가 불안정해진 경우

즉 사용자의 발음 자체는 어느 정도 정확함에도 측정 기준이 지나치게 엄격하거나 노이즈에 취약하여 시각 점수가 의미를 잃는 상황이었습니다.

## 7-2. 한계 ② — Viseme만으로는 판단할 수 없는 음소의 존재

**Viseme**은 입술·턱·치아 등 외부에서 관찰 가능한 조음기관의 형태를 기반으로 정의됩니다.

그러나 영어 발음의 상당수는 혀의 위치, 성대 진동, 공기 흐름 등 외부에서 보이지 않는 요소에 의해 결정되며, 다음과 같은 음소들은 본질적으로 시각 정보만으로는 정답·오답을 판단하기 어려웠습니다.

| 분류        | 해당 음소                             | 시각 판단이 어려운 이유           |
|-----------|-----------------------------------|-------------------------|
| 혀끝 위치 의존  | /r/ · /l/ · /θ/ · /ð/ · /s/ · /z/ | 결정적 단서가 구강 내부 혀의 위치에 있음 |
| 폐쇄음 · 파열음 | /t/ · /d/ · /k/ · /g/             | 입을 다물거나 비슷한 모양만 짧게 보임   |
| 비음        | /n/ · /ŋ/ · /m/                   | 입 다문 형태가 거의 동일해 구분 곤란   |

위 음소들은 사용자가 정확히 발음하더라도 시각 분석에서 일관된 점수를 만들기 어려웠고, 반대로 잘못 발음하더라도 입 모양은 유사할 수 있어 시각 점수 자체의 신뢰도가 음소별로 크게 달라지는 근본적 문제가 존재하였습니다.

### 7-3. 해결 과정

위 두 한계를 동시에 완화하기 위해, 시각 분석 모듈에 다음 두 가지 보정 장치를 단계적으로 도입하였습니다.

#### (1) viseme별 차등 target / tolerance / weight 튜닝

모든 viseme에 동일한 기준값을 적용하던 방식에서 벗어나, viseme별로 결정적인 feature가 다르다는 점에 착안하여 파라미터를 개별 조정하였습니다. 예를 들어 입을 크게 벌리는 aa의 jawOpen은 tolerance를 넓게 허용하고, 입술을 다무는 PP는 mouthClose · lipSealScore 가중치를 높게 설정하는 방식으로 개인차로 인한 0점 수렴 현상을 크게 줄였습니다.

#### (2) visual\_reliability 가중치 도입

모든 viseme에 동일한 가중치를 부여하지 않고, 시각적 구분 가능성에 따라 visual\_reliability  $\in [0, 1]$  값을 부여하였습니다. 입 모양이 뚜렷한 aa, PP, FF, OW 등에는 0.9 수준의 높은 신뢰도를, 시각 정보가 부족한 TH, DD, kk, NN 등에는 0.3 이하의 낮은 신뢰도를 부여하여, 시각으로 판단 불가능한 음소는 시각 점수에 거의 기여하지 않도록 설계하였습니다. 이로써 한계 ②에서 식별한 음소들이 사용자의 실제 발음과 무관하게 점수를 왜곡하는 문제를 구조적으로 해소하였습니다.

### 7-4. 결과

두 가지 보정 장치를 차례로 적용한 이후, 동일한 사용자 발음 샘플에서 시각 점수가 0점에 수렴하던 케이스가 거의 사라졌으며, 시각 정보로 판단 불가능한 음소가 다수 포함된 단어에서도 안정적인 시각 점수가 산출됨을 확인하였습니다. 본 해결 과정은 사전학습 AI

결합형 시스템의 특성상 모델 자체를 재학습할 수 없는 환경에서, 점수 산출 규칙의 설계만으로 신뢰도를 끌어올린 사례라는 점에서 의미를 가집니다.